SOFTWARE DESIGN ARCHITECTURE

President University Lost Center

Lecturer

Hendra Jayanto, M.Sc

Name:

1.	Andri	001202400094
2.	Farrel Vidyano sanggen	001202400128
3.	Hengky Setiawan	001202400198
4.	Michelle	001202400017
5.	Naila Olivia Ramadhani	001202400112
6.	Reza Fahlevi	001202400066

Study Program: IT Cyber Security Class 3

PRESIDENT UNIVERSITY

Jababeka Education Park, Jl. Ki Hajar Dewantara, Kota Jababeka, Cikarang Baru, Bekasi 17550 Indonesia

2026

SOFTWARE DESIGN ARCHITECTURE **1**
I. Introduction **3**
I.1 Purpose 3
I.2 Scope 3
I.3 Intended Audience 3
I.4 Definitions, Acronyms, Abbreviations 4
II. System Overview **4**
II.1 System Goals 4
II.2 User Roles 4
II.3 Functional Overview 5
III. System Architecture Design **6**
III.1 Architecture Pattern 6
III.2 System Architecture Diagram 6
III.3 Data Flow Description 6
System Flow 6
IV. Database Design **7**
IV.1 Database Overview 7
IV.2 Entity Description 7
IV.3 ERD (Entity Relationship Diagram) 8
IV.4 Table Structure 9
V. Module Design **12**
V.1 Authentication Module 12
V.2 Dashboard Module 12
V.3 Report Management Module 12
V.4 Claim Module 12
V.5 Chat and Messaging Module 12
V.6 Notification Module 13
V.7 Email Notification Module 13
V.8 Admin Module 13
VI. UML Diagrams **14**
VI.1 Use Case Diagram 14
VI.4 Class Diagram 19
VII. User Interface Design **21**
VII.1 Register Page 21
VII.2 Login Page 21
VII.3 Dashboard Page 22
VII.4 Report Page 22
VII.5 Chat / Messaging Page 23
VII.6 Notification Page 24
VII.7 Profile Page 24
VIII. Security Design **25**
IX. Technology Stack **26**
X. Conclusion **27**

I. Introduction

I.1 Purpose

The purpose of this Software Design Architecture (SDA) document is to provide a comprehensive and detailed description of the architectural design of the President University Lost Center (PULC) system. This document outlines the system's structure, components, interactions, and design decisions made throughout the development process.

This document serves as a technical reference for developers, system designers, and academic evaluators to understand how the PULC system is structured and how its components interact with one another. It also acts as a guide to ensure that the system is developed in a consistent, maintainable, and scalable manner.

I.2 Scope

The President University Lost Center (PULC) is a web-based application designed to facilitate the reporting, searching, and management of lost and found items within President University. The system provides a centralized digital platform that replaces the previous manual and scattered reporting methods used across different faculties and buildings.

The PULC system allows users to perform the following actions:

- Report lost or found items with supporting images and descriptions
- Browse and search item listings using keywords, categories, and filters
- Submit claim requests for found items and verify ownership
- Communicate directly with item reporters or claimants via an in-app chat feature
- Receive real-time web notifications and email alerts for important events
- Manage reports and users through an administrator dashboard

This application is accessible through any modern web browser without requiring any software installation, ensuring ease of access for all students, staff, and lecturers of President University.

I.3 Intended Audience

This document is intended for:

- **Developers**: To understand the system architecture and implement features accordingly
- **System Designers**: To review and validate the architectural decisions
- **Project Supervisors**: To oversee the overall system design
- **Academic Evaluators (Lecturers)**: To assess the quality and completeness of the software design

I.4 Definitions, Acronyms, Abbreviations

- **SDA**: Software Design Architecture

- **SRS**: Software Requirements Specification
- **PULC**: President University Lost Center
- **UI**: User Interface
- **UX**: User Experience
- **DB**: Database
- **MVC**: Model View Controller
- **API**: Application Programming Interface
- **JWT**: JSON Web Token
- **ORM**: Object Relational Mapping
- **ERD**: Entity Relationship Diagram
- **DBMS**: Database Management System
- **REST**: Representational State Transfer

II. System Overview

The Lost Center System is a web platform that simplifies the process of reporting and finding lost found items by digitizing reporting and searching activities into a single integrated platform.

II.1 System Goals

The President University Lost Center system is designed with the following primary goals:

- To provide a centralized and integrated platform for managing all lost and found item reports within President University
- To improve the efficiency of item searching through structured, categorized, and filterable data
- To ensure secure, reliable, and transparent interaction between users through verification and messaging features
- To reduce the risk of fraud and miscommunication by providing an official verification platform
- To send real-time notifications and email alerts to keep users informed at all times

II.2 User Roles

Guest (Unregistered User)

- Can browse and view lost and found item listings
- Can search and filter items without logging in
- Must register to access advanced features

Registered User (Student / Lecturer / Staff)

- Can register and log into the system

- Can create lost or found item reports
- Can submit claim requests for found items
- Can communicate with other users via the in-app chat feature
- Can receive web notifications and email alerts
- Can manage their own profile

Administrator

- Can manage and moderate all item reports
- Can manage user accounts
- Can update item statuses
- Can monitor overall system activity

II.3 Functional Overview

The system includes the following core functionalities:

- **User Authentication and Authorization**: Secure registration and login using JWT-based authentication
- **Item Reporting**: Users can report lost or found items with title, description, category, location, date, and images
- **Item Search and Filtering**: Users can search items by keyword and filter by category, location, and date
- **Claim Management**: Registered users can submit claim requests and provide verification information
- **Chat and Messaging**: Users can communicate directly through an in-app messaging feature after a claim is initiated
- **Notification System**: Real-time web notifications are triggered by important events such as new messages, claim submissions, and status updates
- **Email Notification Integration**: Automatic email alerts are sent to users via an external email service
- **Admin Control Panel**: Administrators can manage users, moderate reports, and update item statuses

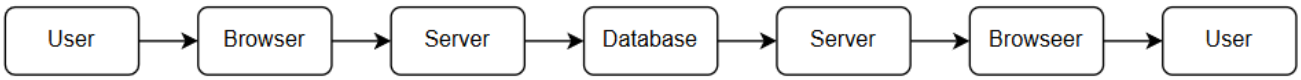
III. System Architecture Design

III.1 Architecture Pattern

The system uses the MVC architectural pattern to ensure a clear separation of concerns and improve maintainability, and support scalable development.

- **Model Layer**: Responsible for handling all data-related logic, including database interactions, data validation, and business rules. In this system, the Model layer is managed by Prisma ORM connected to a PostgreSQL database.
- **View Layer**: Responsible for presenting the user interface to the end user. This layer is implemented using React.js, which renders dynamic and responsive UI components on the client side.
- **Controller Layer**: Acts as the intermediary between the Model and View layers. It handles incoming HTTP requests, processes business logic, and returns appropriate responses. This layer is implemented using Express.js on the server side.

III.2 System Architecture Diagram



The system consist of three main components architecture:

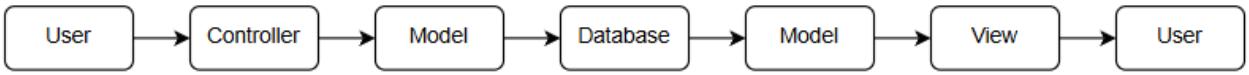
- **Presentation Layer (Client Side):** Handles user interaction through a web browser using HTML,CSS, and JavaScript.
- **Application Layer (Server Side):** Processes business logic using PHP.
- **Data Layer (Database):** Store and manage data using a MySQL.

III.3 Data Flow Description

The system processes data through the following sequence:

- The user sends a request through the web browser
- The request is handled by the controller
- The controller communicates with the model
- The model interacts with the database
- The database returns the requested data
- The model forwards the data to the controller
- The controller passes the data to the view
- The view displays the result to the user

System Flow



IV. Database Design

IV.1 Database Overview

The PULC system uses **PostgreSQL** as its relational database management system. The database is designed using a relational model to ensure data consistency, referential integrity,

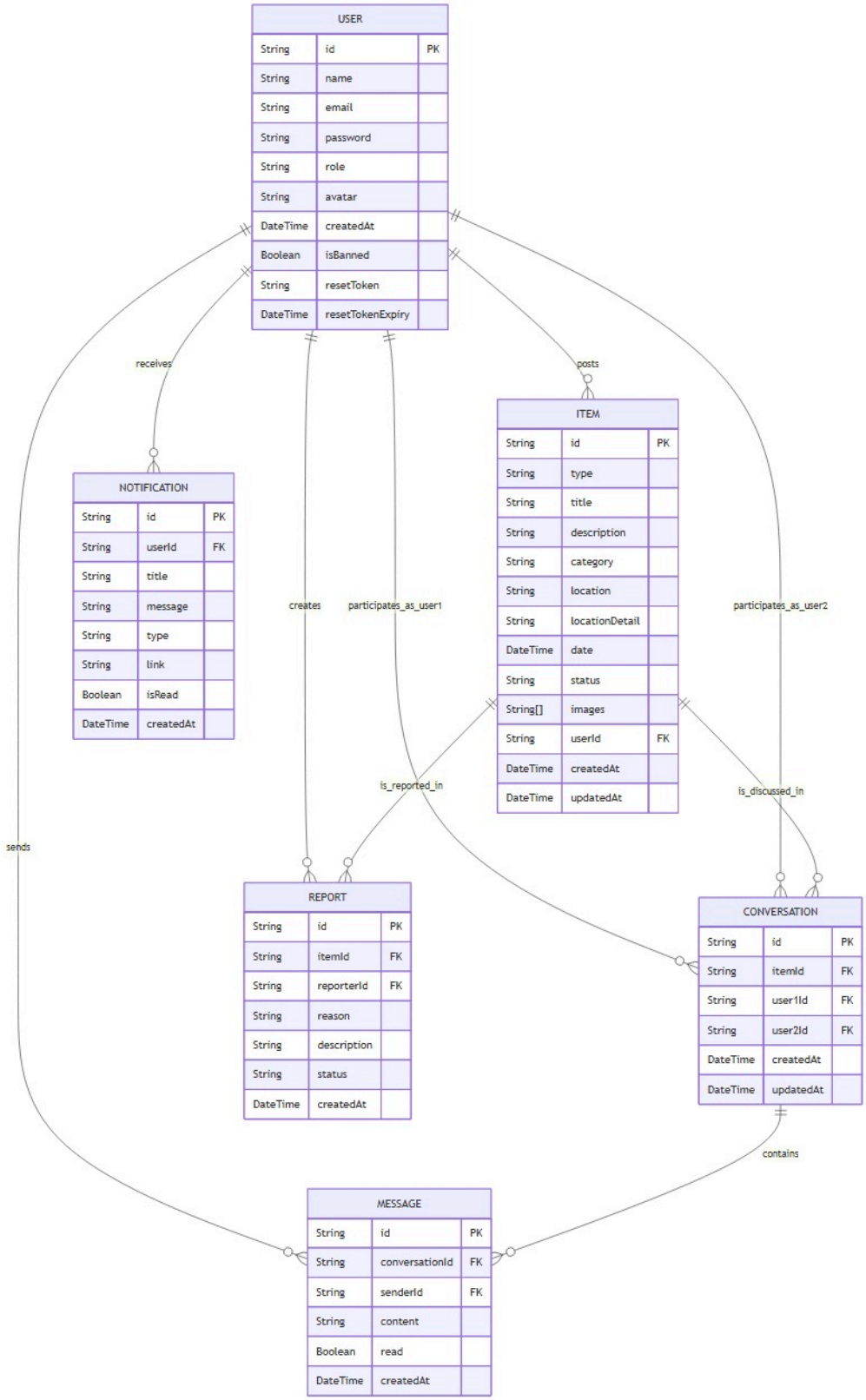
and efficient data retrieval. All database interactions are handled through **Prisma ORM**, which provides type-safe and efficient query execution.

IV.2 Entity Description

The database consists of the following main entities:

- **User**: Stores all user account information, authentication credentials, and role assignments
- **Item**: Stores all information related to lost and found item reports, including type, description, category, location, status, and images
- **Conversation**: Stores conversation sessions initiated between two users in relation to a specific item claim
- **Message**: Stores individual messages exchanged within a conversation
- **Notification**: Stores all in-app notifications generated by system events such as new messages, claim submissions, and status updates

IV.3 ERD (Entity Relationship Diagram)



The relationships between entities are defined as follows:

- One **User** can create multiple **Items** (one-to-many)
- One **User** can receive multiple **Notifications** (one-to-many)
- One **Item** can belong to multiple **Conversations** (one-to-many)
- One **Conversation** contains multiple **Messages** (one-to-many)
- One **Conversation** involves exactly two **Users** (many-to-many via Conversation)

IV.4 Table Structure

Table: Users

Field	Type	Description
id	string	Primary key
username	string	Username
email	string	User email
password	string	Encrypted password
role	string	User role
avatar	string	Profile picture URL
createdAt	DateTime	Account creation timestamp
isBanned	Boolean	Ban status indicator
resetToken	string	Password reset token (Optional, Unique)
resetTokenExpiry	Datetime	Password reset token expiration time

Table: Items

Field	Type	Description
id	string	Primary key
type	string	Item type ("lost" or "found")
title	string	Item title
description	string	Item description
category	string	Item category
location	string	General location
locationDetail	string	Detailed location description

date	string	Date item was lost or found
status	string	Status ("active", "claimed", "resolved")
images	string	Array of image URLs
userId	string	Foreign key referencing Users
createdAt	DateTime	Report creation timestamp
updatedAt	DateTime	Last updated timestamp

Table: Conversations

Field	**Type**	**Description**
id	string	Primary key
ItemId	string	Foreign key referencing Items
user1Id	string	Foreign key referencing Users (initiator)
user2Id	string	Foreign key referencing Users (recipient)
createdAt	DateTime	Conversation creation timestamp
updatedAt	DateTime	Last updated timestamp

Table: Messages

Field	**Type**	**Description**
id	String	Primary key (UUID)
conversationId	string	Foreign key referencing Conversations
senderId	string	Foreign key referencing Users
content	string	Message text content
read	Boolean	Message read status
createdAt	DateTime	Message creation timestamp

Table: notification

Field	Type	Description
id	String	Primary key (UUID)
UserId	string	Foreign key referencing Users
title	string	Notification title
message	string	Notification message bod
type	string	Notification type (e.g., "info")
link	string	Action link (Optional)
IsRead	string	Read status
createdAt	DateTime	Notification creation timestamp

Table: Reports

Field	Type	Description
id	string	Primary key (UUID)
itemId	string	Foreign key referencing reported Items
reporterId	string	Foreign key referencing Users (who reported)
reason	string	Reason for reporting the item
description	string	Additional details/description (Optional)
status	string	Report status (e.g., "pending")
createdAt	datetime	Report creation timestamp

V. Module Design

V.1 Authentication Module

- **Description**: Handles all user registration and login processes, including credential validation and JWT token generation
- **Inputs**: Full name, email, password, role (for registration); email, password (for login)
- **Process**: Validate input fields → Check email existence in database → Hash password (registration) or compare hash (login) → Generate JWT token → Return token to client
- **Outputs**: JWT access token and user session, or error message

V.2 Dashboard Module

- **Description**: Displays all available lost and found item listings with search and filter capabilities
- **Inputs**: User session (optional), search keyword, filter parameters (category, location, date, type)
- **Process**: Fetch all active item listings from database → Apply search and filter queries → Return paginated results
- **Outputs**: List of matching item cards with title, image, category, location, and status

V.3 Report Management Module

- **Description**: Manages the creation, updating, and moderation of lost and found item reports
- **Inputs**: Item title, description, category, location, location detail, date, type (lost/found), images
- **Process**: Validate input fields → Upload images via Multer middleware → Store image paths → Create item record in database → Return success confirmation
- **Outputs**: Newly created item report displayed on the browse page

V.4 Claim Module

- **Description**: Handles the process of submitting and managing item claim requests
- **Inputs**: Item ID, claimant user ID, verification information
- **Process**: Validate claim request → Update item status to "claimed" → Trigger conversation creation → Notify item owner
- **Outputs**: Claim confirmation, updated item status, and initiated conversation

V.5 Chat and Messaging Module

- **Description**: Manages all real-time communication between users within the context of an item claim
- **Inputs**: Conversation ID, sender ID, message content
- **Process**: Validate user participation in conversation → Store message in database → Mark previous messages as read → Return updated conversation
- **Outputs**: Delivered message displayed in the conversation thread

V.6 Notification Module

- **Description**: Generates and manages all in-app notifications triggered by system events
- **Inputs**: Target user ID, notification type, title, content
- **Process**: Trigger notification event → Create notification record in database → Push real-time web notification to target user → Mark notification as read upon user interaction
- **Outputs**: Real-time web notification displayed to the target user

V.7 Email Notification Module

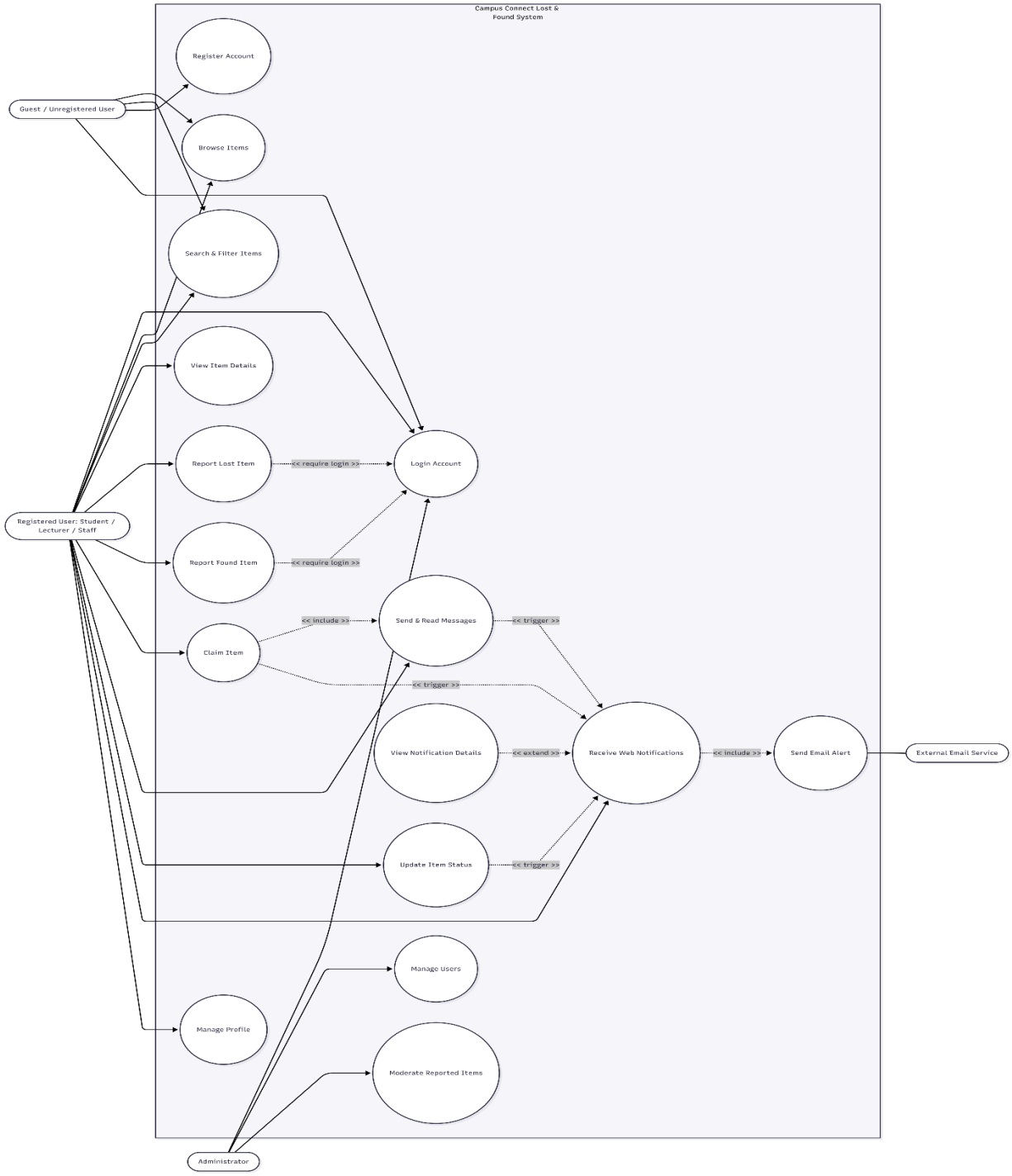
- **Description**: Sends automatic email alerts to users when important events occur, ensuring they are informed even when offline
- **Inputs**: Target user email, notification subject, notification body
- **Process**: Trigger email event → Format email content → Send via external email service (EmailService interface)
- **Outputs**: Email delivered to the target user's inbox

V.8 Admin Module

- **Description**: Provides administrators with full control over system management, including user management, report moderation, and status updates
- **Inputs**: Admin session, target user or item ID, action type
- **Process**: Verify admin role → Execute management action (update, delete, moderate) → Log activity → Return updated system state
- **Outputs**: Updated user list, moderated item reports, or updated item statuses

VI. UML Diagrams

VI.1 Use Case Diagram



The Use Case Diagram illustrates the interactions between three main actors and the system:

Guest / Unregistered User can perform the following actions:

- Register Account
- Browse Items
- Search and Filter Items

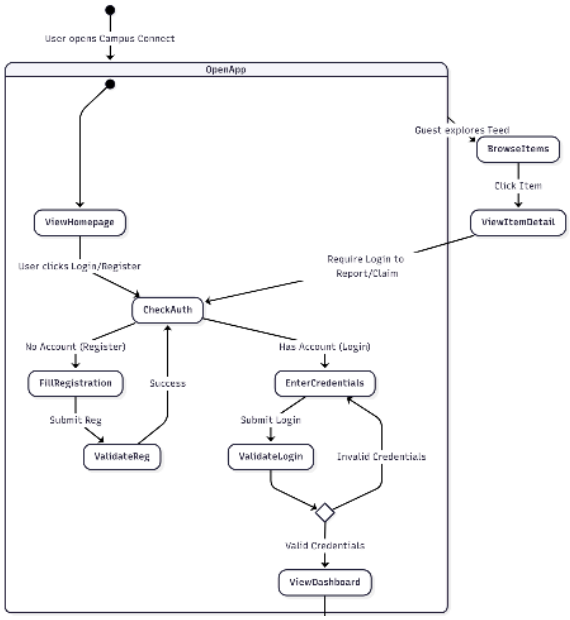
Registered User (Student / Lecturer / Staff) can perform the following actions:

- All Guest actions
- Login to Account
- Report Lost Item
- Report Found Item
- View Item Details
- Claim Item
- Send and Read Messages
- View Notification Details
- Receive Web Notifications
- Update Item Status
- Manage Profile

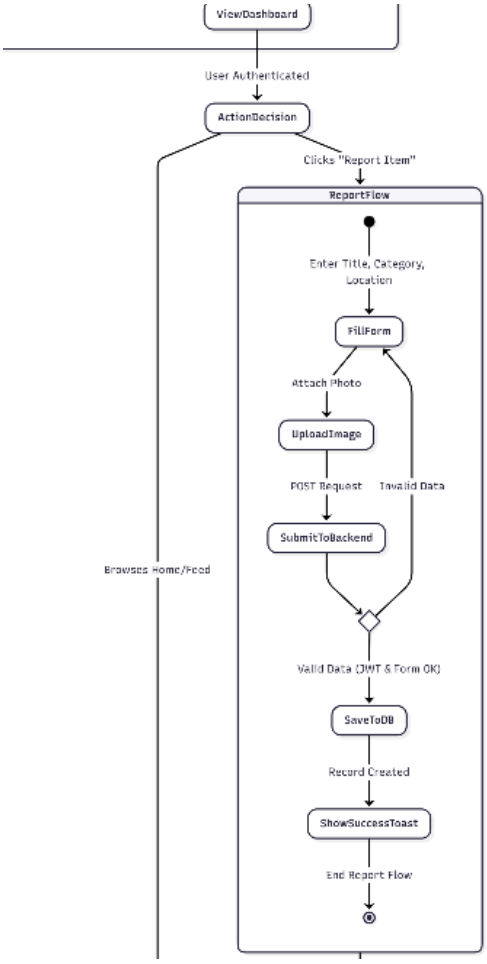
Administrator can perform the following actions:

- All Registered User actions
- Manage Users
- Moderate Reported Items

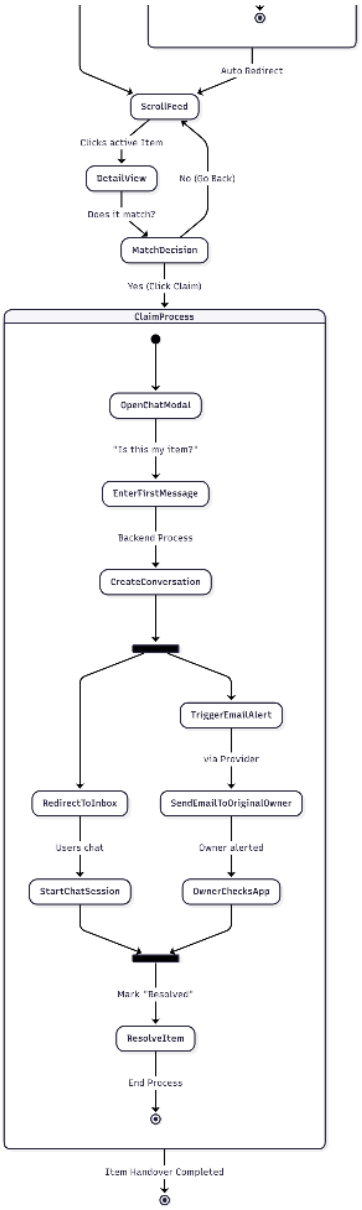
VI.2 Activity Diagram



(1)

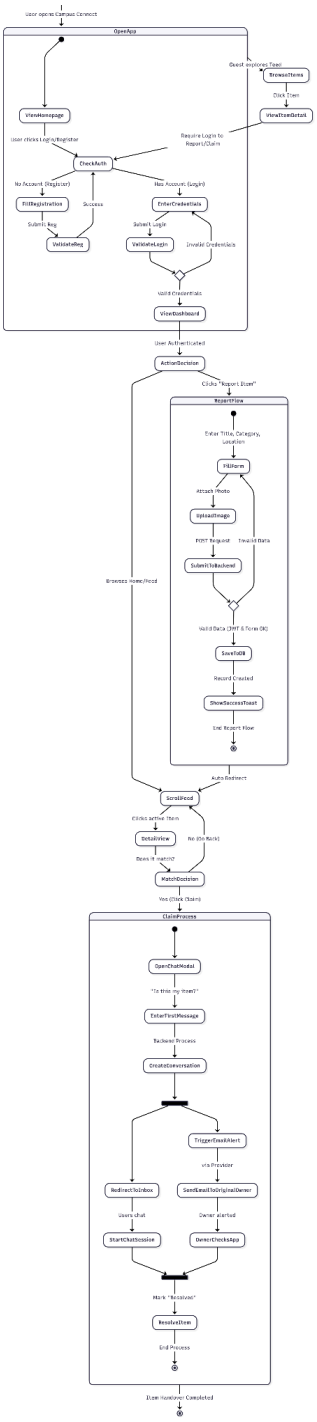


(2)



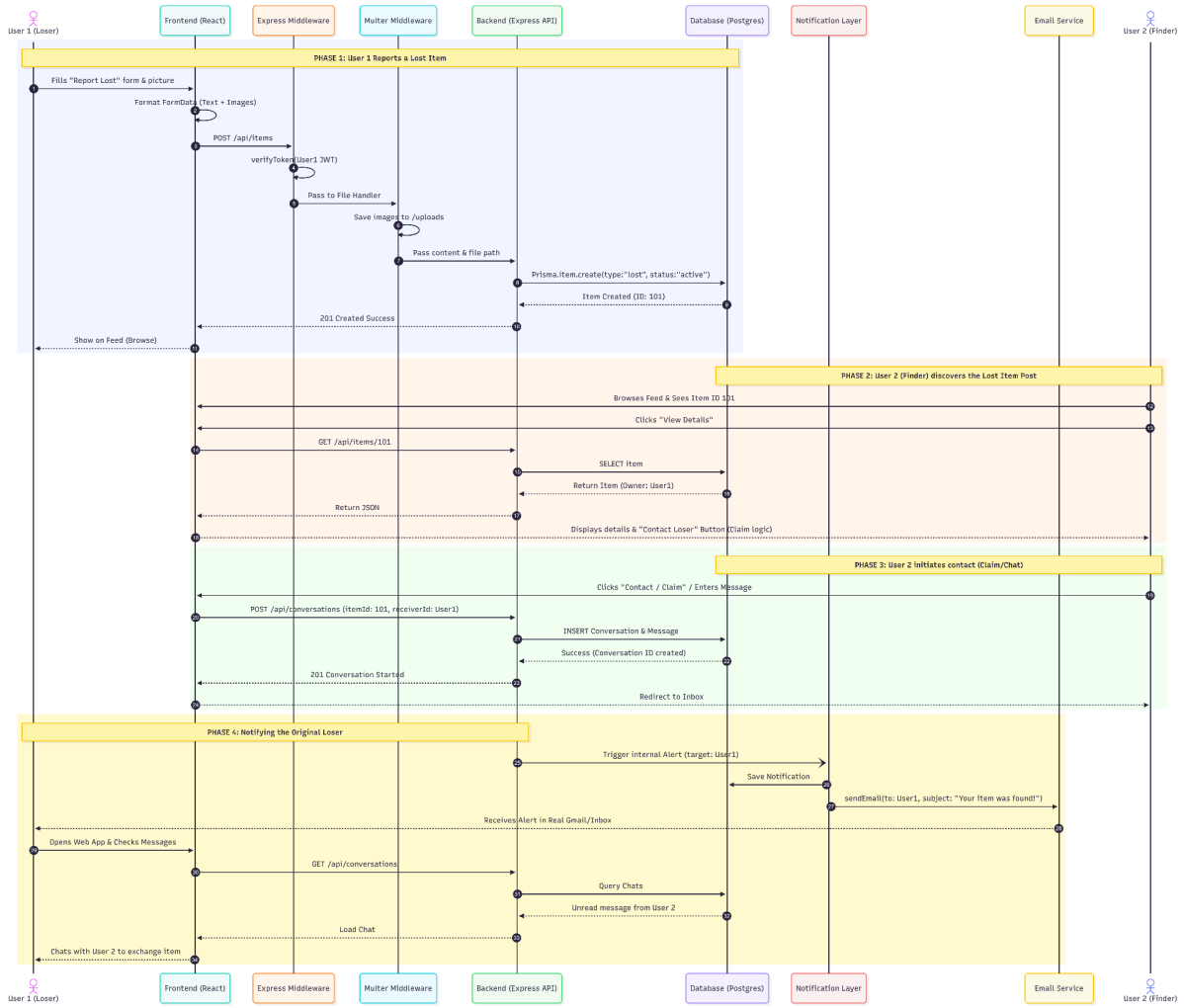
((3)

The Activity Diagram describes the complete workflow of the PULC system, from the moment a user accesses the platform to the resolution of an item claim. The flow includes user authentication, item reporting, item browsing, claim submission, chat initiation, and notification delivery.



(Full version)

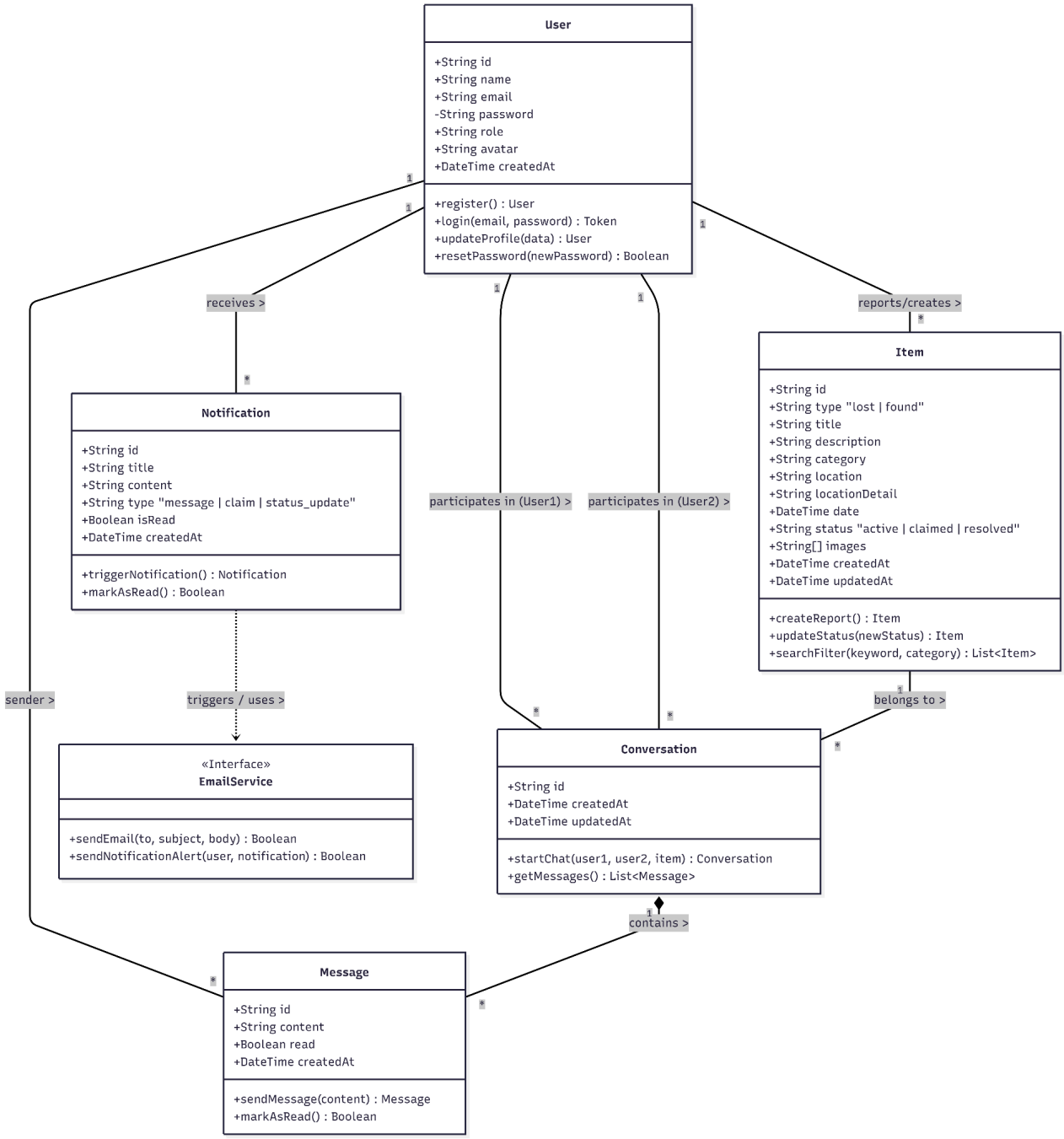
VI.3 Sequence Diagram



The Sequence Diagram illustrates the interaction between system components across four main phases:

- **Phase 1**: User 1 (Loser) reports a lost item through the frontend, which sends a POST request to the backend API. The Multer middleware handles image uploads, and Prisma ORM creates the item record in the PostgreSQL database.
- **Phase 2**: User 2 (Finder) discovers the lost item by browsing the platform and clicks "View Details" to retrieve complete item information via a GET request.
- **Phase 3**: User 2 initiates contact by clicking "Contact / Claim", which triggers a POST request to create a new conversation and sends an initial message. The system redirects both users to the chat inbox.
- **Phase 4**: The system notifies User 1 (the original loser) by triggering an internal notification alert, sending a real-time web notification, and delivering an email alert via the external Email Service

VI.4 Class Diagram



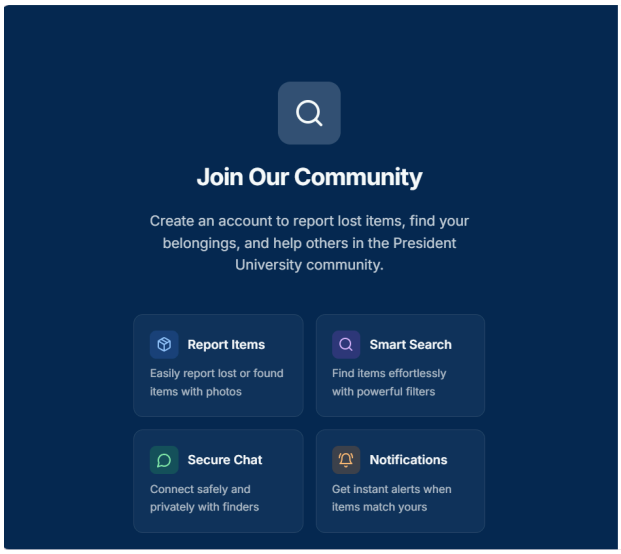
The Class Diagram illustrates the structure and relationships of the system's main classes:

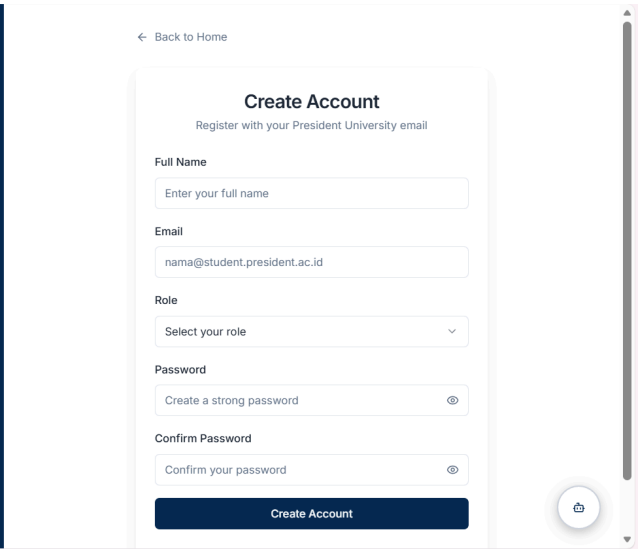
- **User**: Contains attributes such as id, name, email, password, role, avatar, and createdAt. Methods include register(), login(), updateProfile(), and resetPassword().
- **Item**: Contains attributes such as id, type, title, description, category, location, locationDetail, date, status, images, createdAt, and updatedAt. Methods include createReport(), updateStatus(), and searchFilter().
- **Conversation**: Contains attributes such as id, createdAt, and updatedAt. Methods include startChat() and getMessages().
- **Message**: Contains attributes such as id, content, read, and createdAt. Methods include sendMessage() and markAsRead().
- **Notification**: Contains attributes such as id, title, content, type, isRead, and createdAt. Methods include triggerNotification() and markAsRead().
- **EmailService (Interface)**: Defines the contract for email operations including sendEmail() and sendNotificationAlert().

VII. User Interface Design

VII.1 Register Page

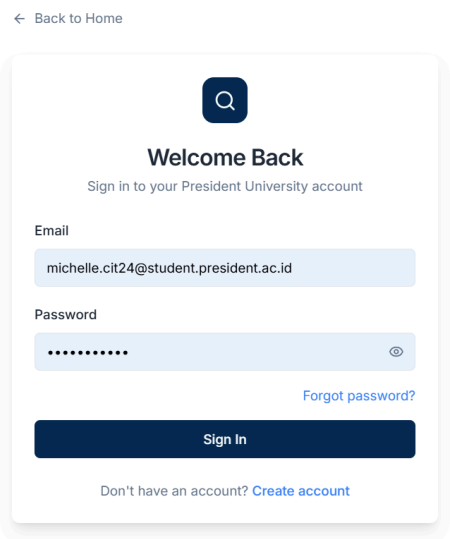
The Register Page allows new users to create an account on the PULC platform using their President University email address.

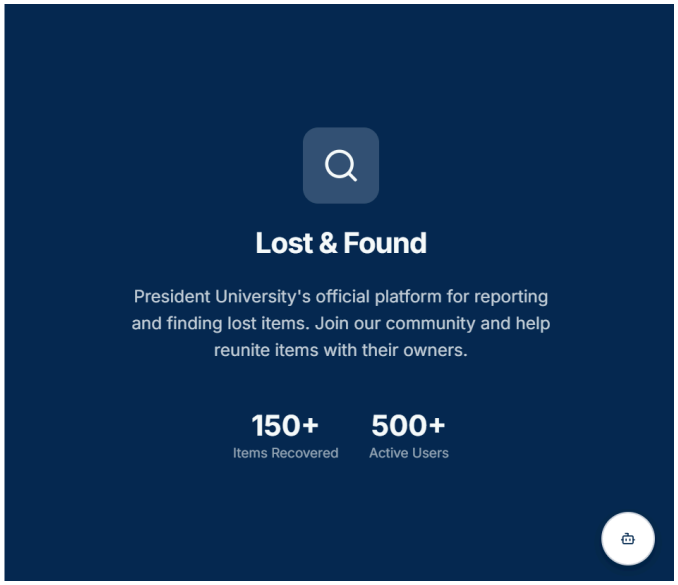




VII.2 Login Page

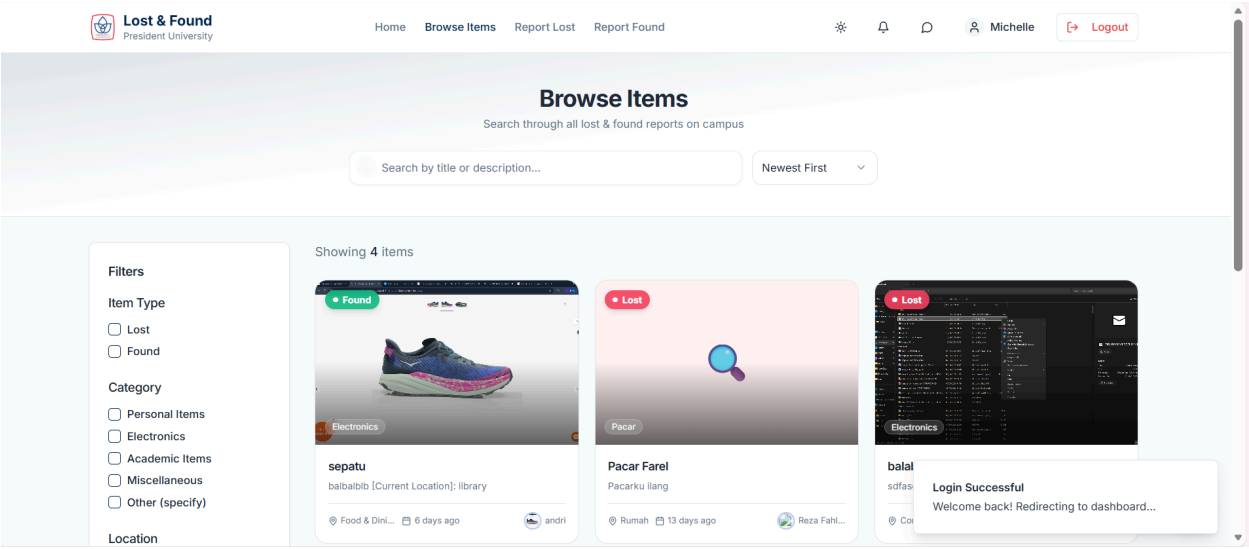
The Login Page allows registered users to authenticate and access the system.





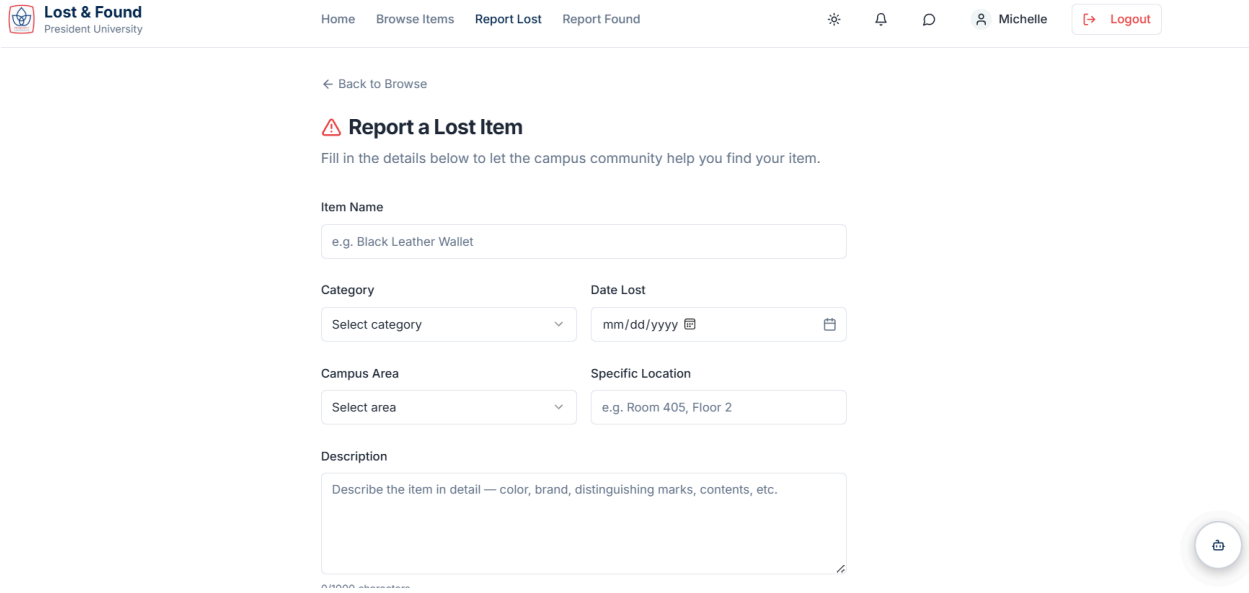
VII.3 Dashboard Page

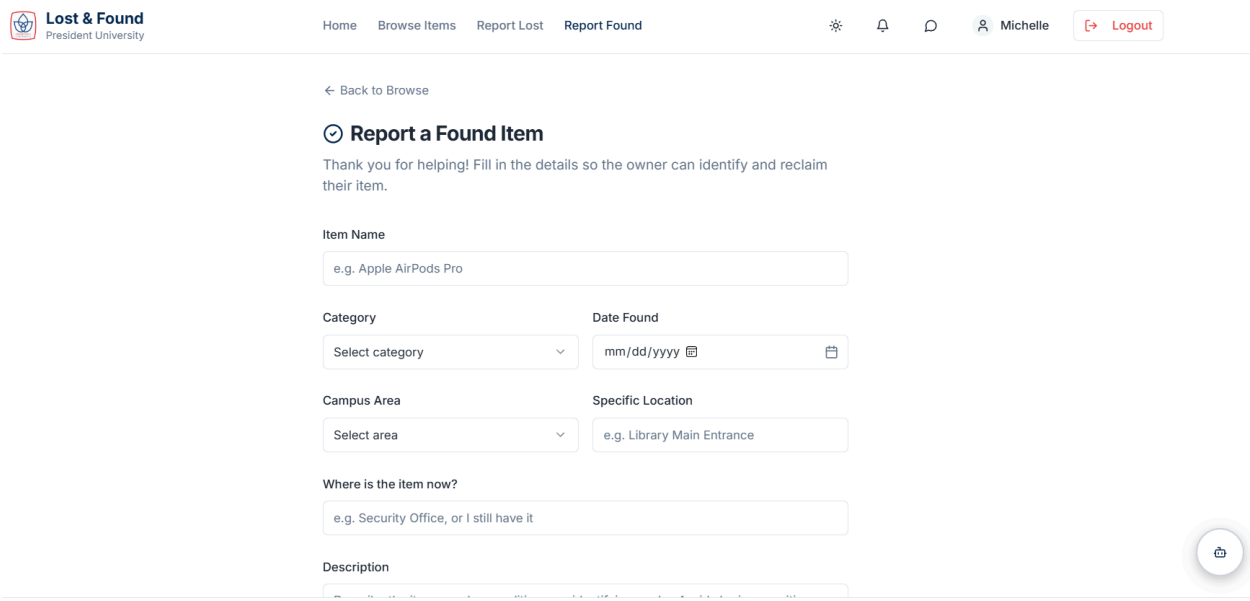
Displays system overview, navigation menu, and user options.



VII.4 Report Page

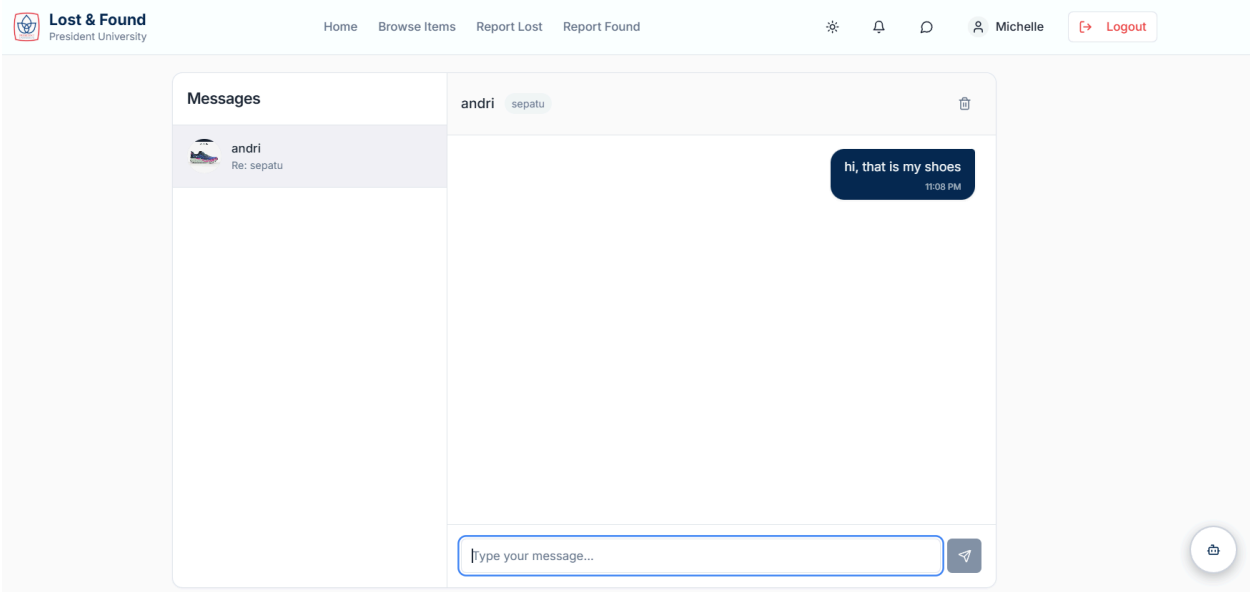
Allows users to submit lost or found item reports with details such as title, description, and category. The newly created report then appears on the Dashboard Page for other users to view.





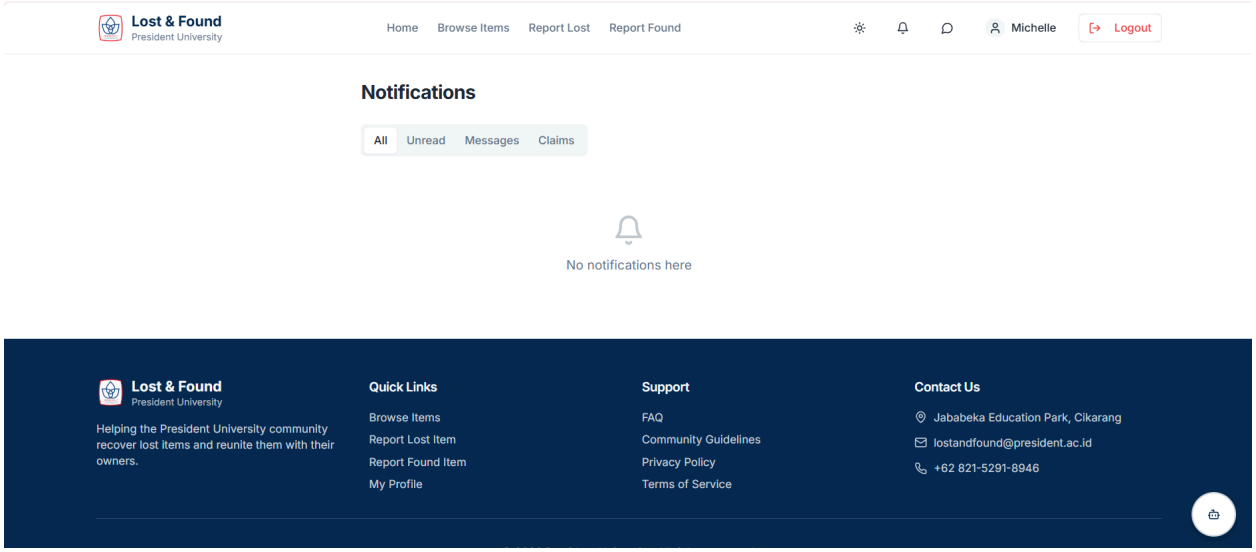
VII.5 Chat / Messaging Page

The Chat Page provides a direct messaging interface between two users involved in an item claim.



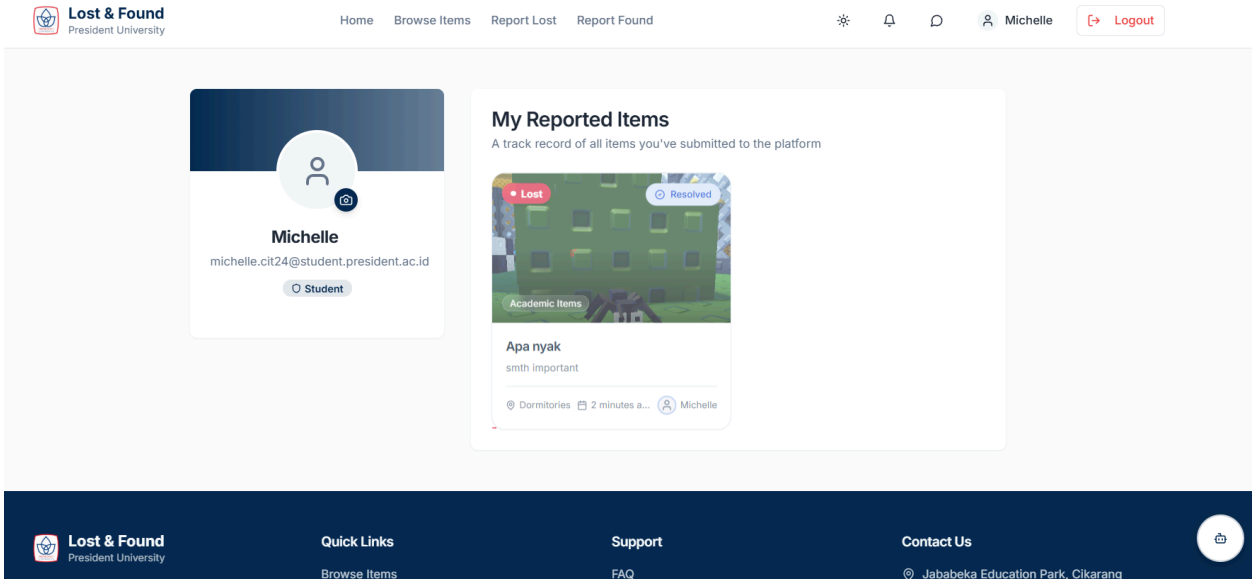
VII.6 Notification Page

The Notification Page displays all in-app notifications received by the logged-in user.



VII.7 Profile Page

The Profile Page allows users to view and manage their personal account information, including their full name, email address, profile picture, and role.



VIII. Security Design

The PULC system implements several security measures to protect user data and maintain overall system integrity throughout all layers of the application.

- **Password Hashing**: All user passwords are encrypted using a cryptographic hashing algorithm (bcrypt) before being stored in the database. This ensures that plaintext passwords are never stored or exposed, even in the event of a database breach.
- **JWT-Based Authentication**: The system uses JSON Web Tokens (JWT) for user authentication. Upon successful login, a JWT token is generated and sent to the client. This token must be included in the header of every subsequent API request that requires authentication. The server validates the token on each request to ensure the user is authorized.
- **Input Validation and Sanitization**: All user inputs are validated and sanitized on both the client side (React.js) and server side (Express.js) to prevent common web vulnerabilities such as SQL injection and Cross-Site Scripting (XSS).
- **Role-Based Access Control (RBAC)**: The system enforces role-based access control to restrict certain functionalities to authorized users only. For example, admin-specific routes and actions are protected and inaccessible to regular users.
- **File Upload Security**: Image uploads are handled by the Multer middleware, which enforces file type restrictions and size limits to prevent the upload of malicious files.
- **HTTPS Communication**: All data transmitted between the client and server is encrypted using HTTPS, protecting sensitive information from being intercepted during transmission.

IX. Technology Stack

The PULC system is built using the following technologies:

Frontend

- React.js : Main frontend framework for building dynamic and component-based UI
- React Router: Client-side routing and navigation management
- Framer Motion: Animation library for smooth UI transitions and interactions

Backend

- Express.js : Main backend framework for handling HTTP requests and RESTful API routing
- Express Middleware: Request processing, authentication validation, and error handling
- Multer Middleware: File upload handling for item images

Database

- PostgreSQL: Relational database management system for structured data storage
- Prisma ORM:Type-safe database access and query management

Hosting and Deployment

- Vercel : Cloud hosting platform for both frontend and backend deployment

Development Tools

- Visual Studio Code: Primary code editor
- Git & GitHub : Version control and collaborative development
- Postman : API testing and documentation

X. Conclusion

The President University Lost Center (PULC) system is designed using a well-structured, modular, and layered software architecture to ensure scalability, maintainability, and a high-quality user experience. The adoption of the MVC architectural pattern, combined with a clearly separated three-tier client-server structure, supports efficient development, easier debugging, and the ability to extend the system with new features in the future.

The use of modern and industry-standard technologies — including React.js for the frontend, Express.js for the backend, PostgreSQL for the database, and Prisma ORM for data management — ensures that the system is both robust and developer-friendly. Security is prioritized at every layer of the system through JWT-based authentication, password hashing, input validation, role-based access control, and HTTPS communication.

The PULC system successfully addresses the core problems of the previous manual lost and found process at President University by providing a centralized, accessible, and efficient digital platform. With features such as item reporting, advanced search and filtering, claim management, in-app messaging, and real-time notifications, the system significantly improves the experience of reporting and recovering lost items for all members of the university community.